

DAY 10 — DUCKDB + STRUCTURED ANALYTICS + ENGINEERING QUERY ENGINE

GOAL

Move from:

RAG search 

to:

structured engineering analytics 

WHY DAY 10 IS IMPORTANT

Current system:

LLM retrieval works 

BUT:

LLM is weak for exact calculations 

aggregations 

comparisons 

engineering analytics 

Example:

Which condition has highest brake temperature?

Average CDT across reports?

Compare GHD vs HSD?

LLM alone is unreliable.

DAY 10 FIXES

 DuckDB structured storage

 SQL querying

 engineering analytics

 exact numeric calculations

 metadata-aware filtering

✓ hybrid RAG + SQL

✓ DAY 10 ARCHITECTURE

PDF

↓

Extraction

↓

Tables

↓

Chunks

↓

ChromaDB ✓ semantic

AND

↓

Structured Records

↓

DuckDB ✓ analytics

✓ TEMPLATE STRUCTURE

app/

└─ storage/

| └─ duckdb_store.py ✓

|

└─ analytics/

| └─ analytics_engine.py ✓

|

└─ query/

| └─ sql_query.py ✓

|

└─ ingestion/

| └─ pipeline.py ✓ MODIFY

✓ STEP 1 — INSTALL

pip install duckdb

✓ STEP 2 — DUCKDB STORAGE

 **app/storage/duckdb_store.py**

import duckdb

```
conn = duckdb.connect(
    "engineering.db"
)
```

```
def initialize_db():
```

```
    conn.execute("""
```

```
    CREATE TABLE IF NOT EXISTS engineering_chunks (
```

```
        chunk_id TEXT,
```

```
        page INTEGER,
```

```
        table_id TEXT,
```

```
        content TEXT
```

```
    )
```

```
    """)
```

```
def insert_chunks(chunks):

    for chunk in chunks:

        conn.execute("""
        INSERT INTO engineering_chunks
        VALUES (?, ?, ?, ?)
        """, (

            chunk["id"],

            chunk["metadata"]["page"],

            chunk["metadata"]["table_id"],

            chunk["text"]
        ))
```

✅ STEP 3 — STORE INTO DUCKDB

 **app/ingestion/pipeline.py**

Add:

```
from app.storage.duckdb_store import (
    initialize_db,
    insert_chunks
)
```

Add before return:

```
initialize_db()
```

```
insert_chunks(all_chunks)
```

✓ STEP 4 — SQL QUERY ENGINE

 **app/query/sql_query.py**

```
from app.storage.duckdb_store import (  
    conn  
)
```

```
def run_sql(query):
```

```
    result = conn.execute(query)
```

```
    return result.fetchall()
```

✓ STEP 5 — ANALYTICS ENGINE

 **app/analytics/analytics_engine.py**

```
from app.query.sql_query import (  
    run_sql  
)
```

```
def get_temperature_rows():
```

```
    query = """
```

```
    SELECT *
```

```
    FROM engineering_chunks
```

```
    WHERE LOWER(content)
```

```
    LIKE '%temperature%'
```

```
"""
```

```
return run_sql(query)
```

✅ STEP 6 — TEST SQL

 **tests/test_sql.py**

```
from app.query.sql_query import (  
    run_sql  
)
```

```
result = run_sql("""
```

```
SELECT *
```

```
FROM engineering_chunks
```

```
LIMIT 10
```

```
""")
```

```
print(result)
```

✅ STEP 7 — TEST ANALYTICS

 **tests/test_analytics.py**

```
from app.analytics.analytics_engine import (  
    get_temperature_rows  
)
```

```
results = get_temperature_rows()
```

for r in results:

```
print(r)
```

✔ WHAT DAY 10 ACHIEVES

- ✔ exact SQL filtering
 - ✔ engineering analytics
 - ✔ numeric querying
 - ✔ scalable storage
 - ✔ metadata filtering
 - ✔ hybrid SQL + RAG
-

✔ HUGE ARCHITECTURE SHIFT

Before:

LLM-only retrieval

Now:

Semantic AI ✔

+

Structured Analytics ✔

✔ CURRENT SYSTEM

PDF

↓

Extraction

↓

Normalization

↓

Filtering

↓

Embeddings → ChromaDB 

AND

↓

Structured Storage → DuckDB 

EXAMPLE FUTURE QUERIES

SELECT *

FROM engineering_chunks

WHERE content LIKE '%Maximum%'

SELECT COUNT(*)

FROM engineering_chunks

SELECT *

FROM engineering_chunks

WHERE page = 6

WHY THIS IS POWERFUL

Now your system supports:

 semantic retrieval

 exact retrieval

 analytics

 engineering computations

 reporting

 dashboarding

✅ **YOU ARE NOW HERE**

✅ DAY 10 COMPLETE

Next:

DAY 11

→ LangGraph agent

→ tool calling

→ SQL + RAG orchestration

→ engineering copilot workflows